# MATERIAL DE APOYO ESTÁNDARES DE CODIFICACIÓN

## INTRODUCCIÓN

En el desarrollo de software participan diferentes personas, tecnologías y herramientas. Para garantizar que el código pueda ser comprendido, mantenido y evolucionado por diferentes desarrolladores, es necesario establecer **estándares de codificación**.

Un estándar de codificación es un conjunto de reglas y convenciones que orientan la forma en que se escribe, organiza, documenta y mantiene el código fuente de una aplicación.

Estos estándares permiten, entre otros aspectos:

- Mejorar la legibilidad del código.
- Facilitar el mantenimiento de las aplicaciones.
- Reducir errores derivados de interpretaciones diferentes.
- Facilitar el trabajo colaborativo.
- Estandarizar los proyectos de software.
- Facilitar las revisiones de código.
- Mejorar la incorporación de nuevos integrantes al equipo.
- Facilitar la aplicación de herramientas de análisis estático.
- Favorecer la calidad y evolución del software.

## CONCEPTOS FUNDAMENTALES

### ¿Qué es un estándar de codificación?

Es un conjunto de reglas que establece cómo debe escribirse y organizarse el código de un proyecto.

Por ejemplo, un equipo puede establecer:

```
nombre_cliente = "Carlos"
```

en lugar de:

```
nombreCliente = "Carlos"
```

Si el equipo utiliza Python, puede adoptar snake_case como convención para variables y funciones.

Lo importante no es solamente escoger una convención, sino que **todo el equipo la utilice de manera consistente**.

## PRINCIPIOS GENERALES DE NOMENCLATURA

Un buen nombre debe ser:

- Claro.
- Descriptivo.
- Consistente.
- Fácil de leer.
- Fácil de recordar.
- Relacionado con el propósito del elemento.
- Preferiblemente sin ambigüedad.

### Ejemplo incorrecto

```
x = 25
y = 10
z = x * y
```

El código funciona, pero no comunica claramente qué representan los datos.

### Ejemplo mejorado

```
ancho = 25
alto = 10
area = ancho * alto
```

El segundo código requiere menos explicación porque los nombres expresan el significado.

## NOMBRAMIENTO DE VARIABLES

Una variable representa un dato cuyo valor puede cambiar durante la ejecución del programa.

### Convención recomendada

Para Python se recomienda utilizar:

**snake_case**

Las palabras se escriben en minúsculas y se separan mediante guiones bajos.

### Ejemplos

```
nombre_cliente
fecha_nacimiento
precio_producto
cantidad_productos
total_factura
```

### Evitar

NombreCliente
nombreCliente
nombre-cliente
nombre cliente
NC
x
dato1

No significa que nombres como x sean siempre incorrectos. En determinados contextos, por ejemplo en una expresión matemática o un ciclo corto, pueden ser apropiados.

Los nombres deben representar el significado

### Incorrecto

```
d = 20
```

### Mejor

```
dias_vencimiento = 20
```

### Incorrecto

```
p = 50000
```

### Mejor

```
precio_producto = 50000
```

**Variables booleanas**

Las variables booleanas pueden comenzar con palabras que indiquen una condición.

Ejemplos:

```
es_activo = True
esta_pagado = False
tiene_permiso = True
puede_editar = False
```

Esto permite comprender rápidamente que la variable representa una condición.

**EJERCICIO EJEMPLO 1: VARIABLES**

### Situación

Se tiene el siguiente código:

```
n = "Juan"
e = 25
c = 150000
a = e * 12
```

### Actividad

Identifique qué representa cada variable y refactorice el código aplicando un estándar.

### Solución propuesta

```
nombre_cliente = "Juan"
edad_cliente = 25
salario_mensual = 150000
salario_anual = salario_mensual * 12
```

### Reflexión

El programa realiza las mismas operaciones, pero la segunda versión es más fácil de comprender.

## NOMBRAMIENTO DE CONSTANTES

Una constante representa un valor que conceptualmente no debería cambiar durante la ejecución del programa.

En Python, una convención ampliamente utilizada es escribir las constantes en:

**MAYÚSCULAS_CON_GUION_BAJO**

### Ejemplos

```
IVA = 0.19
MAX_INTENTOS = 3
PORCENTAJE_DESCUENTO = 0.10
TIEMPO_SESION = 3600
```

En otros lenguajes pueden existir mecanismos específicos para garantizar que un valor sea inmutable.

Ejemplo

### Sin estándar

```
iva = 0.19
maxIntentos = 3
```

### Aplicando estándar

```
IVA = 0.19
MAX_INTENTOS = 3
```

## EJERCICIO EJEMPLO 2: CONSTANTES

Observe:

precio = 100000
iva = 0.19
maxIntentos = 3
tiempoSesion = 3600

### Actividad

Identifique cuáles representan constantes y modifique sus nombres.

### Solución

IVA = 0.19
MAX_INTENTOS = 3
TIEMPO_SESION = 3600

precio = 100000

## NOMBRAMIENTO DE MÉTODOS O FUNCIONES

Una función o método representa una acción o comportamiento.

Por esta razón, su nombre debe utilizar preferiblemente un **verbo que describa la acción realizada**.

En Python se recomienda snake_case.

### Ejemplos

registrar_cliente()
consultar_producto()
calcular_total()
actualizar_usuario()
eliminar_producto()
validar_credenciales()
generar_factura()

### Evitar

cliente()
producto()
hacer()
datos()
funcion1()
metodo2()

Los nombres anteriores no comunican claramente qué realiza el método.

**Una función debe expresar su responsabilidad**

**Poco descriptivo**

```
def procesar():
    ...
```

**Mejor**

```
def calcular_total_factura():
    ...
```

**Otro ejemplo**

```
def validar_usuario():
    ...
```

es más comprensible que:

```
def usuario():
    ...
```

**EJERCICIO EJEMPLO 3: FUNCIONES**

Analice:

```
def cliente(nombre, edad):
    if edad >= 18:
        return True
    return False
```

**Problema**

El nombre cliente() no representa claramente la operación.

**Refactorización**

```
def validar_mayoria_edad(nombre, edad):
    if edad >= 18:
        return True
    return False
```

Incluso podría simplificarse:

```
def validar_mayoria_edad(edad):
    return edad >= 18
```

La función ahora expresa claramente su responsabilidad.

## NOMBRAMIENTO DE CLASES

Las clases representan entidades, conceptos o comportamientos del dominio de una aplicación.

En Python se recomienda utilizar:

**PascalCase**

Cada palabra comienza con mayúscula y no se utilizan guiones bajos.

### Ejemplos

Cliente
Producto
Factura
Usuario
Pedido
DetalleFactura
GestionUsuarios
ConexionBaseDatos

### Evitar

cliente
cliente_producto
CLASECLIENTE
clase1
datos

## EJEMPLO DE CLASE

### Incorrecto

```
class cliente_producto:
    pass
```

### Correcto

```
class ClienteProducto:
    pass
```

Otro ejemplo:

```
class Producto:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio
```

## RESUMEN DE NOMENCLATURA EN PYTHON

| Elemento | Convención | Ejemplo |
|---|---|---|
| Variable | snake_case | nombre_cliente |
| Constante | MAYÚSCULAS | MAX_INTENTOS |
| Función | snake_case | calcular_total() |
| Método | snake_case | actualizar_cliente() |
| Clase | PascalCase | Cliente |
| Módulo | snake_case | gestion_clientes.py |
| Paquete | minúsculas | clientes |

## NOMBRAMIENTO DE BASES DE DATOS

Los nombres de las bases de datos también deben seguir una convención definida por el equipo.

Una posibilidad es utilizar:

- Minúsculas.
- Palabras separadas por guion bajo.
- Nombres descriptivos.
- Evitar espacios.
- Evitar caracteres especiales.
- Evitar nombres ambiguos.

### Ejemplos

tienda_virtual
gestion_clinica
sistema_reservas
gestion_aprendices

### Evitar

Tienda Virtual
BD1
baseNueva
Base de Datos
sistema-final

## NOMBRAMIENTO DE TABLAS

Aunque el tema central es el nombre de las bases de datos, los estándares también deben extenderse a las tablas.

Una convención posible es utilizar nombres en plural:

clientes
productos
facturas
pedidos
usuarios

Otra organización puede utilizar singular:

cliente
producto
factura
pedido
usuario

Lo importante es **seleccionar una convención y utilizarla consistentemente en todo el proyecto**.

## NOMBRES DE CAMPOS DE BASES DE DATOS

Los nombres de los campos deben ser claros, descriptivos y consistentes.

### Ejemplo

**Tabla**: clientes

**Campos**:

id_cliente
nombre
apellido
correo
telefono
fecha_registro
estado

**Campos booleanos:**Puede utilizarse una convención que permita identificar fácilmente valores verdadero/falso.

Ejemplo:

es_activo
tiene_descuento
permite_edicion

**Fechas:**Los nombres deben indicar claramente qué representa la fecha.

**Evitar**
fecha

**Mejor**
fecha_nacimiento
fecha_registro
fecha_pago
fecha_vencimiento

## EJEMPLO DE BASE DE DATOS

**Nombre**
tienda_virtual

**Tabla**
productos

**Campos**
id_producto
codigo
nombre
descripcion
precio
cantidad_stock
fecha_registro
es_activo

Este esquema permite identificar fácilmente el propósito de cada elemento.

## DOCUMENTACIÓN DE MÉTODOS

La documentación permite explicar:

- Qué hace un método.
- Qué parámetros recibe.
- Qué devuelve.
- Qué excepciones puede generar.
- Qué condiciones deben cumplirse.
- Qué efectos secundarios puede producir.

Una función bien documentada facilita el mantenimiento y el trabajo colaborativo.

## COMENTARIOS VS. DOCUMENTACIÓN

No todo el código necesita comentarios.

### Comentario

Explica generalmente una parte específica de la implementación:

```
# Convertir el precio a decimal
precio = Decimal(precio)
```

### Documentación

Describe formalmente el propósito y comportamiento de una función.

En Python se utilizan comúnmente **docstrings**.

Ejemplo:

```
def calcular_total(precio, cantidad):
    """
    Calcula el valor total de un producto.

    Args:
        precio (float): Precio unitario del producto.
        cantidad (int): Cantidad de unidades.

    Returns:
        float: Valor total de la compra.
    """
    return precio * cantidad
```

## ELEMENTOS DE UNA DOCUMENTACIÓN DE MÉTODO

Dependiendo del lenguaje y estándar seleccionado, la documentación puede incluir:

### 1. Descripción

¿Qué hace el método?

### 2. Parámetros

¿Qué información recibe?

### 3. Retorno

¿Qué información devuelve?

### 4. Excepciones

¿Qué errores puede producir?

### 5. Precondiciones

¿Qué debe cumplirse antes de ejecutar el método?

### 6. Ejemplo

¿Cómo se utiliza?

## EJEMPLO COMPLETO DE DOCUMENTACIÓN

```
def calcular_descuento(precio, porcentaje):
    """
    Calcula el valor del descuento aplicado a un producto.

    Args:
        precio (float): Precio del producto.
        porcentaje (float): Porcentaje de descuento entre 0 y 100.

    Returns:
        float: Valor correspondiente al descuento.

    Raises:
        ValueError: Si el precio es negativo o el porcentaje
                    está fuera del rango permitido.
    """

    if precio < 0:
        raise ValueError("El precio no puede ser negativo")

    if porcentaje < 0 or porcentaje > 100:
        raise ValueError("El porcentaje debe estar entre 0 y 100")

    return precio * porcentaje / 100
```

## BUENAS PRÁCTICAS GENERALES

Los aprendices deben considerar las siguientes recomendaciones:

**1. Preferir nombres descriptivos**
cantidad_productos

en lugar de:

cp

**2. Evitar nombres excesivamente largos**

Un nombre debe ser descriptivo sin convertirse en una frase completa.

**3. Mantener consistencia**

No utilizar simultáneamente:

nombre_cliente
nombreCliente
NombreCliente

dentro del mismo proyecto.

**4. Evitar abreviaturas innecesarias**

Preferir: cantidad_productos

sobre: cant_prod

**5. Utilizar verbos en funciones**
registrar_usuario()
eliminar_producto()
consultar_factura()

**6. Utilizar sustantivos en clases**
Usuario
Producto
Factura
Cliente

**7. Documentar lo que aporte información útil**

La documentación debe explicar el propósito y contrato del código, evitando comentarios que simplemente repitan lo que el código ya expresa.

## CONCLUSIÓN

Los estándares de codificación constituyen una herramienta fundamental para desarrollar software mantenible y facilitar el trabajo colaborativo.

Un código funcional no necesariamente es un código de calidad. Dos programas pueden producir exactamente el mismo resultado, pero uno puede ser mucho más fácil de comprender, modificar, probar y mantener.

Por esta razón, el desarrollador debe adquirir el hábito de utilizar convenciones consistentes para:

- Variables.
- Constantes.
- Funciones.
- Métodos.
- Clases.
- Bases de datos.
- Campos.
- Documentación.

La aplicación de estos estándares debe realizarse desde las primeras etapas del proyecto y mantenerse durante todo el ciclo de vida del software.

## REFERENCIA RÁPIDA DEL ESTÁNDAR

| **Elemento** | **Convención sugerida** | **Ejemplo** |
|---|---|---|
| Variable | snake_case | nombre_cliente |
| Constante | UPPER_SNAKE_CASE | MAX_INTENTOS |
| Función | Verbo + snake_case | registrar_cliente() |
| Método | Verbo + snake_case | actualizar_producto() |
| Clase | PascalCase | Cliente |
| Base de datos | snake_case | tienda_virtual |
| Tabla | Convención uniforme | producto |
| Campo | snake_case | fecha_registro |
| Booleano | Prefijo semántico | es_activo |
| Documentación | Docstring/documentación estándar | """Descripción..""" |

**Nota:** Las convenciones anteriores son una propuesta para el proceso formativo. En un proyecto profesional deben adaptarse a las convenciones del lenguaje, framework, organización y equipo de desarrollo. Lo fundamental es establecerlas explícitamente y aplicarlas de manera consistente.

**Referencias:**


1. Una guía completa sobre estándares de codificación para calidad y cumplimiento: https://es.parasoft.com/learning-center/coding-standards/
2. Qué son los estandares de codificación: https://blog.codacy.com/coding-standards
3. Estandares de Codificación: https://www.youtube.com/watch?v=F8H3xnI4RIs
4. Estándares de código y buenas prácticas en programación: https://www.youtube.com/watch?v=DxMxA4wpbJ8


**CONTROL DEL DOCUMENTO**

| | Nombre | Cargo | Dependencia | Fecha |
|---|---|---|---|---|
| **Autor (es)** | César Marino Cuéllar Chacón | Instructor | CTPI-CAUCA | 02-11-2026 |